

Modul 10 Shell

Tujuan Praktikum

1. Praktikan mampu memahami shell pada Xinu.
2. Praktikan mampu mengimplementasikan perintah baru pada Shell.

10.1 Shell

1. Pada modul sebelumnya telah dijelaskan mengenai syscall (layanan yang disediakan oleh sistem operasi). Akan tetapi, user normal tidak akan pernah/jarang mengakses syscall secara langsung. Biasanya user mengakses aplikasi dan aplikasi tersebut yang akan mengakses syscall.
2. Shell adalah **program** yang memproses perintah yang diberikan oleh user pada terminal. Shell akan looping secara terus menerus membaca baris input yang diberikan. Setelah sebuah baris input dibaca ditandai dengan adanya ENTER, shell harus mengekstrak nama perintah, argumen dan hal-hal lainnya. Jika proses ekstraksi berhasil maka perintah akan dieksekusi sesuai dengan argumen yang diberikan.

Contoh perintah pada shell:

ls -al

nama_perintah = "ls"
argument 1 = "-al"

3. Cara kerja shell dapat dipelajari pada source code ./shell/shell.c.

```
/* Print shell banner and startup message */  
fprintf(dev, "\n\n%s%s\n%s\n%s\n%s\n%s\n%s\n%s\n%s\n%s\n",  
SHELL_BAN0,SHELL_BAN1,SHELL_BAN2,SHELL_BAN3,SHELL_BAN4,  
SHELL_BAN5,SHELL_BAN6,SHELL_BAN7,SHELL_BAN8,SHELL_BAN9);  
  
fprintf(dev, "%s\n\n", SHELL_STRMSG);  
  
/* Continually prompt the user, read input, and execute command */  
while (TRUE) {  
  
    /* Display prompt */  
    fprintf(dev, SHELL_PROMPT);  
  
    /* Read a command */  
    len = read(dev, buf, sizeof(buf));  
  
    /* Exit gracefully on end-of-file */  
    if (len == EOF) {  
        break;  
    }  
  
    /* If line contains only NEWLINE, go to next line */  
    if (len <= 1) {  
        continue;  
    }  
  
    buf[len] = SH_NEWLINE; /* terminate line */  
  
    /* Parse input line and divide into tokens */  
}
```

Gambar 10-1 Shell.c

4. Perintah-perintah Linux (shell command) berada pada file system/harddisk (contoh: perintah ls berada pada direktori /bin/ls). Pada Xinu, shell command berada pada image (yaitu image yang dibuat pada kompilasi Xinu). Jadi untuk menambahkan atau mengurangi shell command (perintah pada shell), Xinu harus dicompile ulang seluruhnya.

5. Berikut adalah cara menambahkan perintah baru “mycmd” pada Xinu

a. Edit shell.c yang berada pada direktori xinu/shell/

Tambahkan {“mycmd”, FALSE, xsh_mycmd}, pada struct cmdent cmdtab[]

```

/*****
/* Table of Xinu shell commands and the function associated with each */
*****/
const struct cmdent cmdtab[] = {
    {"argecho",    TRUE,  xsh_argecho},
    {"arp",        FALSE, xsh_arp},
    {"cat",        FALSE, xsh_cat},
    {"clear",      TRUE,  xsh_clear},
    {"date",       FALSE, xsh_date},
    {"devdump",    FALSE, xsh_devdump},
    {"echo",       FALSE, xsh_echo},
    {"exit",       TRUE,  xsh_exit},
    {"help",       FALSE, xsh_help},
    {"kill",       TRUE,  xsh_kill},
    {"memdump",    FALSE, xsh_memdump},
    {"memstat",    FALSE, xsh_memstat},
    {"netinfo",    FALSE, xsh_netinfo},
    {"ping",       FALSE, xsh_ping},
    {"ps",         FALSE, xsh_ps},
    {"sleep",      FALSE, xsh_sleep},
    {"udp",        FALSE, xsh_udpdump},
    {"udpecho",    FALSE, xsh_udpecho},
    {"udpserver",  FALSE, xsh_udpserver},
    {"uptime",     FALSE, xsh_uptime},
    {"mycmd",      FALSE, xsh_mycmd},
    {"?",         FALSE, xsh_help}
};

```

Gambar 10-2 Struct cmdent cmdtab[]

b. Edit “shprototype.h” yang berada pada folder xinu/include/
 Tambahkan extern shellcmd xsh_mycmd (int32, char *[]);

```

/* in file xsh_sleep.c */
extern shellcmd xsh_sleep      (int32, char *[]);

/* in file xsh_udpdump.c */
extern shellcmd xsh_udpdump    (int32, char *[]);

/* in file xsh_udpecho.c */
extern shellcmd xsh_udpecho     (int32, char *[]);

/* in file xsh_udpserver.c */
extern shellcmd xsh_udpserver   (int32, char *[]);

/* in file xsh_uptime.c */
extern shellcmd xsh_uptime      (int32, char *[]);

/* in file xsh_help.c */
extern shellcmd xsh_help        (int32, char *[]);

extern shellcmd xsh_mycmd       (int32, char *[]);

```

Gambar 10-3 Penambahan extern shellcmd xsh_mycmd Pada shprototype.h

- c. Buat file baru bernama xsh_mycmd.c pada folder xinu/shell

```

/* xsh_mycmd.c - xsh_mycmd */

#include <xinu.h>
#include <stdio.h>
#include <string.h>
// deklarasi nama fungsi
static void printMemUse(void);

shellcmd xsh_mycmd(int nargs, char *args[]) {

    /* For argument '--help', emit help about the 'ping' command */
    // mycmd --help
    // jika perintah adalah "mycmd --help" tampilkan help
    if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
        printf("Use: %s argumentdd1 argumen2\n\n", args[0]);
        printf("Description:\n");
        printf("\tMy own command with 2 argument\n");
        printf("Options:\n");
        printf("\t--help\t display this help and exit\n");
        return 0;
    }

    /* Check for valid number of arguments */
    // banyaknya argument harus 2; args[0] = nama_perintah args[1]= argumen 1 args[2] = argumen 2
    if (nargs != 3) {
        fprintf(stderr, "%s: jumlah argumen tidak cocok\n", args[0]);
        fprintf(stderr, "Try '%s --help' for more information\n",
                args[0]);
        return 1;
    }

    printMemUse();
    printf("Nama perintah adalah: %s \n", args[0]);
    printf("Argumen pertama adalah: %s \n", args[1]); // 123
    printf("Argumen kedua adalah: %s \n", args[2]); // xxx
    return 0;
}

/*-----
 * printMemUse - Print statistics about memory use
 *-----
*/

```

Gambar 10-4 xsh_mycmd.c

```

/*-----
 * printMemUse - Print statistics about memory use
 *-----
 */
void printMemUse(void){
    printf("My memory usage is xxxx\n");
}

```

Gambar 10-5 Fungsi printMemUse

- d. Compile ulang Xinu

Pada development-system lakukan hal-hal berikut ini:

 - cd /home/xinu/xinu/compile
 - make clean
 - make
 - sudo minicom
- e. Test hasil perintah baru pada shell
 - xsh \$ mycmd
 - xsh \$ mycmd 111
 - xsh \$ mycmd 111 xxx



Fakultas Informatika
 School of Computing
 Telkom University

